

Deployment Guide

This guide covers deploying your E-Commerce Automotive Store to production. We'll focus on Vercel (recommended) and Railway as deployment platforms.

Table of Contents

1. [Pre-Deployment Checklist](#)
 2. [Vercel Deployment \(Recommended\)](#)
 3. [Railway Deployment](#)
 4. [Database Setup](#)
 5. [Environment Variables](#)
 6. [Custom Domain Setup](#)
 7. [SSL Certificate](#)
 8. [Post-Deployment Checklist](#)
 9. [Monitoring & Maintenance](#)
-

Pre-Deployment Checklist

Before deploying to production, ensure you have:

Code Preparation

- ☐ All bugs fixed and tested locally
- ☐ Removed all `console.log` statements and debug code
- ☐ Environment variables properly configured
- ☐ Database migrations tested
- ☐ Test data removed from database
- ☐ All placeholder content replaced (yourdomain.com, etc.)
- ☐ Error handling implemented
- ☐ Loading states added for all async operations

Assets

- ☐ Images optimized (compressed, proper formats)
- ☐ Favicon and PWA icons generated
- ☐ Logo files added
- ☐ All static assets in `public/` directory

Security

- ☐ Strong passwords set for database
- ☐ Stripe live API keys ready (not test keys)
- ☐ NEXTAUTH_SECRET generated and secure
- ☐ CORS settings configured properly
- ☐ Rate limiting implemented (if needed)

SEO

- ☐ Meta tags and descriptions updated
- ☐ Sitemap generated
- ☐ robots.txt configured
- ☐ Open Graph images created
- ☐ Analytics tracking code added

Legal

- ☐ Terms & Conditions page completed
- ☐ Privacy Policy page completed
- ☐ Return/Refund policy page completed
- ☐ Cookie consent implemented (if required)

Vercel Deployment (Recommended)

Vercel is the recommended platform for Next.js applications. It offers excellent performance, automatic scaling, and easy setup.

Why Vercel?

- ✓ Built by Next.js creators
- ✓ Zero configuration for Next.js
- ✓ Automatic HTTPS
- ✓ Global CDN
- ✓ Serverless functions
- ✓ Free hobby plan available
- ✓ Easy database integration

Step 1: Create Vercel Account

1. Go to <https://vercel.com>
2. Click “Sign Up”
3. Sign up with GitHub, GitLab, or Bitbucket

Step 2: Import Your Project

Option A: Deploy from GitHub

1. **Push your code to GitHub:**

```
bash
git add .
git commit -m "Ready for deployment"
git push origin main
```

2. **Import to Vercel:**

- Go to <https://vercel.com/new>
- Click “Import Git Repository”
- Select your repository
- Click “Import”

Option B: Deploy with Vercel CLI

1. Install Vercel CLI:

```
bash
npm install -g vercel
```

2. Login to Vercel:

```
bash
vercel login
```

3. Deploy:

```
bash
vercel
```

4. Follow the prompts:

- Set up and deploy: `Y`
- Which scope: Select your account
- Link to existing project: `N`
- Project name: `your-store-name`
- Directory: `./` (default)
- Override settings: `N`

Step 3: Configure Build Settings

In the Vercel dashboard:

1. Go to your project
2. Click "Settings" → "General"
3. Configure:
 - **Framework Preset:** Next.js
 - **Build Command:** `npm run build`
 - **Output Directory:** `.next` (default)
 - **Install Command:** `npm install`
 - **Node Version:** 18.x or higher

Step 4: Set Up Database (Vercel Postgres)

Option A: Vercel Postgres

1. In your Vercel project dashboard
2. Click "Storage" tab
3. Click "Create Database"
4. Select "Postgres"
5. Choose region (closest to your users)
6. Click "Create"
7. Vercel automatically adds `DATABASE_URL` to your environment variables

Option B: External Database (Supabase, Neon, etc.)

See [Database Setup](#) section below.

Step 5: Configure Environment Variables

1. In Vercel dashboard, go to "Settings" → "Environment Variables"
2. Add all variables from your `.env.local` :

Variable	Value	Environment
NEXTAUTH_SECRET	[generated secret]	Production, Preview
NEXTAUTH_URL	https://yourdomain.com	Production
NEXTAUTH_URL	https://your-project.vercel.app	Preview
NEXT_PUBLIC_API_URL	https://yourdomain.com	Production
DATABASE_URL	[your database URL]	Production, Preview
STRIPE_SECRET_KEY	sk_live_...	Production
STRIPE_SECRET_KEY	sk_test_...	Preview
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY	pk_live_...	Production
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY	pk_test_...	Preview
STRIPE_WEBHOOK_SECRET	whsec_...	Production, Preview
EMAIL_HOST	[SMTP host]	Production, Preview
EMAIL_PORT	587	Production, Preview
EMAIL_USER	[your email]	Production, Preview
EMAIL_PASS	[app password]	Production, Preview
EMAIL_FROM	noreply@yourdomain.com	Production, Preview

1. Click “Save” for each variable

Step 6: Deploy

1. Trigger deployment:

- Push to GitHub: Vercel auto-deploys
- Or click “Deployments” → “Redeploy”

2. Monitor deployment:

- Watch build logs in real-time
- Fix any errors that appear

3. Deployment complete:

- Your site is live at `https://your-project.vercel.app`
- Visit the URL to test

Step 7: Run Database Migrations

After first deployment:

```
# Connect to production database
psql [your-production-database-url]

# Run migrations
\i scripts/schema.sql

# Or use migration tool
npx prisma migrate deploy # if using Prisma
```

Railway Deployment

Railway is a simpler alternative that includes database hosting.

Step 1: Create Railway Account

1. Go to <https://railway.app>
2. Sign up with GitHub
3. Verify your email

Step 2: Create New Project

1. Click “New Project”
2. Select “Deploy from GitHub repo”
3. Select your repository
4. Click “Deploy Now”

Step 3: Add PostgreSQL Database

1. In your project dashboard
2. Click “New”
3. Select “Database” → “PostgreSQL”
4. Railway creates database and adds `DATABASE_URL` automatically

Step 4: Configure Environment Variables

1. Click on your service (not database)
2. Go to “Variables” tab
3. Click “Raw Editor”
4. Paste all environment variables:

```
env
NEXTAUTH_SECRET=your_secret
NEXTAUTH_URL=${{RAILWAY_PUBLIC_DOMAIN}}
NEXT_PUBLIC_API_URL=${{RAILWAY_PUBLIC_DOMAIN}}
DATABASE_URL=${{Postgres.DATABASE_URL}}
STRIPE_SECRET_KEY=sk_live...
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY=pk_live...
STRIPE_WEBHOOK_SECRET=whsec...
EMAIL_HOST=smtp.gmail.com
```

```
EMAIL_PORT=587
EMAIL_USER=your@email.com
EMAIL_PASS=your_app_password
EMAIL_FROM=noreply@yourdomain.com
```

1. Click “Update Variables”

Step 5: Configure Build Settings

Railway auto-detects Next.js, but verify:

1. Go to “Settings” tab
2. Check:
 - **Build Command:** `npm run build`
 - **Start Command:** `npm start`
 - **Port:** 3000 (auto-detected)

Step 6: Deploy

1. Railway automatically deploys
2. Monitor build logs
3. Fix any errors
4. Visit generated URL (e.g., `https://your-app.up.railway.app`)

Database Setup

Option 1: Vercel Postgres

Pros: Integrated, automatic configuration, serverless

Cons: Limited free tier

1. Already covered in Vercel deployment section above
2. Connection string automatically added to environment variables

Option 2: Supabase

Pros: Generous free tier, additional features (auth, storage, realtime)

Cons: Slightly more setup

1. Go to `https://supabase.com`
2. Create new project
3. Wait for database to provision (~2 minutes)
4. Go to “Project Settings” → “Database”
5. Copy “Connection string” (URI format)
6. Add to environment variables:

```
env
```

```
DATABASE_URL=postgresql://postgres:[password]@db.[project-ref].supabase.co:5432/postgres
```

Option 3: Neon

Pros: Serverless Postgres, autoscaling, free tier

Cons: Newer platform

1. Go to `https://neon.tech`

2. Create account and new project
3. Copy connection string
4. Add to environment variables

Option 4: Railway Postgres

Already covered in Railway deployment section.

Database Migration

After setting up your production database:

```
# Method 1: Using psql
psql [your-production-database-url] -f scripts/schema.sql

# Method 2: Using Prisma (if using Prisma ORM)
npx prisma migrate deploy

# Method 3: Using Drizzle (if using Drizzle ORM)
npx drizzle-kit push:pg
```

 **Important:** Never run seed data in production!

Environment Variables

Production vs Preview Environments

Set different values for production and preview:

Production (yourdomain.com):

- Stripe live keys (`sk_live_...` , `pk_live_...`)
- Production database
- Production email settings

Preview (vercel.app subdomains):

- Stripe test keys (`sk_test_...` , `pk_test_...`)
- Development/staging database
- Test email settings

Required Environment Variables

```
# Authentication
NEXTAUTH_SECRET=[generate new: openssl rand -base64 32]
NEXTAUTH_URL=https://yourdomain.com
NEXT_PUBLIC_API_URL=https://yourdomain.com

# Database
DATABASE_URL=postgresql://user:pass@host:5432/dbname

# Stripe - PRODUCTION
STRIPE_SECRET_KEY=sk_live_[your_key]
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY=pk_live_[your_key]
STRIPE_WEBHOOK_SECRET=whsec_[your_secret]

# Email
EMAIL_HOST=smtp.gmail.com
EMAIL_PORT=587
EMAIL_USER=your@email.com
EMAIL_PASS=[app_password]
EMAIL_FROM=noreply@yourdomain.com

# Cloudinary (if used)
CLOUDINARY_API_KEY=[your_key]
CLOUDINARY_API_SECRET=[your_secret]

# Analytics (optional)
NEXT_PUBLIC_GA_ID=[google_analytics_id]
```

Custom Domain Setup

Vercel Custom Domain

1. Add domain in Vercel:

- Go to project "Settings" → "Domains"
- Enter your domain: `yourdomain.com`
- Click "Add"

2. Configure DNS:

Option A: Using Vercel nameservers (recommended)

- Vercel provides nameservers
- Update nameservers at your domain registrar

Option B: Using A/CNAME records

...

Type: A

Name: @

Value: 76.76.19.19

Type: CNAME

Name: www

Value: cname.vercel-dns.com

...

1. **Add www subdomain:**

- Add `www.yourdomain.com` as another domain
- Vercel automatically redirects `www` → `non-www` (or vice versa)

2. **Wait for propagation:**

- DNS changes take 1-48 hours
- Check status in Vercel dashboard

Railway Custom Domain

1. Go to “Settings” → “Domains”
2. Click “Add Custom Domain”
3. Enter your domain
4. Add CNAME record at your DNS provider:

Type: CNAME

Name: @

Value: [provided by Railway]

SSL Certificate

Vercel

☒ **Automatic HTTPS** - SSL certificates are automatically provisioned and renewed
No configuration needed!

Railway

☒ **Automatic HTTPS** - SSL certificates are automatically provisioned
No configuration needed!

Manual SSL (if self-hosting)

If deploying to your own server:

1. Use **Let's Encrypt** (free)
2. Use **Certbot** for automatic renewal
3. Configure your web server (Nginx/Apache)

Post-Deployment Checklist

After successful deployment, verify everything works:

Functionality Tests

- [] Homepage loads correctly
- [] All pages accessible (about, products, etc.)
- [] Product pages display properly
- [] Search functionality works

- ☐ Shopping cart adds/removes items
- ☐ Checkout flow completes
- ☐ Stripe payment processing works (use test card first!)
- ☐ Order confirmation email received
- ☐ Admin panel accessible
- ☐ User authentication works
- ☐ Password reset works

Performance Tests

- ☐ Lighthouse score (aim for >90)
- ☐ Page load time <3 seconds
- ☐ Images load quickly
- ☐ No console errors
- ☐ Mobile performance good

SEO Checks

- ☐ Meta tags present on all pages
- ☐ Open Graph images working
- ☐ Sitemap accessible (/sitemap.xml)
- ☐ robots.txt accessible (/robots.txt)
- ☐ SSL certificate valid (HTTPS)
- ☐ Google Search Console submitted

Security Checks

- ☐ HTTPS enabled
- ☐ Environment variables not exposed
- ☐ Admin routes protected
- ☐ API routes secured
- ☐ CORS configured properly
- ☐ Rate limiting active

Stripe Production Setup

- ☐ Switched to live API keys
- ☐ Webhook endpoint configured
- ☐ Test live transaction
- ☐ Payout method configured
- ☐ Business details completed in Stripe

Analytics & Monitoring

- ☐ Google Analytics tracking
 - ☐ Vercel Analytics enabled
 - ☐ Error tracking set up (Sentry)
 - ☐ Uptime monitoring configured
-

Monitoring & Maintenance

Vercel Analytics

1. Go to your project → “Analytics” tab
2. Enable Vercel Analytics
3. Monitor:
 - Page views
 - Unique visitors
 - Top pages
 - Performance metrics

Error Tracking (Sentry)

1. **Create Sentry account:** <https://sentry.io>
2. **Install Sentry:**

```
bash
```

```
npm install @sentry/nextjs
```

3. **Initialize Sentry:**

```
bash
```

```
npx @sentry/wizard -i nextjs
```

4. **Add to environment variables:**

```
env
```

```
NEXT_PUBLIC_SENTRY_DSN=your_dsn
```

Uptime Monitoring

Use services like:

- **UptimeRobot** (free) - <https://uptimerobot.com>
- **Pingdom** - <https://pingdom.com>
- **Better Uptime** - <https://betteruptime.com>

Database Backups

Vercel Postgres:

- Automatic backups included
- Point-in-time recovery available

Supabase:

- Daily automatic backups
- Manual backup via dashboard

Railway:

- Manual backups via CLI
- Set up automated backup script

Regular Maintenance

Weekly:

- [] Check error logs
- [] Monitor site performance
- [] Review failed transactions

Monthly:

- [] Update dependencies: `npm update`
- [] Review analytics data
- [] Check for security updates
- [] Test backup restoration

Quarterly:

- [] Major version updates
- [] Security audit
- [] Performance optimization review

Troubleshooting Production Issues

Build Fails

Error: `Module not found`

- Check all imports are correct
- Ensure all dependencies in `package.json`
- Clear cache: `npm clean cache --force`

Error: `Environment variable not found`

- Verify all required variables set in platform
- Check variable names match exactly (case-sensitive)

Database Connection Errors

Error: `Connection refused`

- Check `DATABASE_URL` is correct
- Verify database is running
- Check IP whitelist settings

Error: `SSL required`

- Add `?sslmode=require` to connection string
- Or configure SSL in database settings

Stripe Webhook Not Working

- Verify webhook URL is correct: `https://yourdomain.com/api/checkout/webhook`
- Check `STRIPE_WEBHOOK_SECRET` matches Stripe dashboard
- Review webhook logs in Stripe dashboard
- Ensure API route is not middleware-blocked

Slow Performance

- Enable caching in Next.js config
- Optimize images (use `next/image`)
- Enable Vercel Edge Caching
- Use CDN for static assets
- Optimize database queries

404 on Custom Domain

- Wait for DNS propagation (up to 48 hours)

- Verify DNS records are correct
 - Check domain configuration in platform
 - Try clearing DNS cache: `ipconfig /flushdns`
-

Rollback Procedure

If deployment has critical issues:

Vercel

1. Go to “Deployments”
2. Find last working deployment
3. Click “...” → “Promote to Production”

Railway

1. Go to “Deployments”
 2. Select previous deployment
 3. Click “Redeploy”
-

Scaling Considerations

As your store grows:

Database Scaling

- Monitor query performance
- Add indexes for frequently queried fields
- Consider read replicas for heavy traffic
- Implement caching (Redis)

File Storage

- Move images to CDN (Cloudinary, Imgix)
- Use object storage (AWS S3, Vercel Blob)

Application Scaling

- Vercel/Railway automatically scale
 - Monitor serverless function limits
 - Consider edge computing for global users
-

Support Resources

- **Vercel Documentation:** <https://vercel.com/docs>
- **Next.js Documentation:** <https://nextjs.org/docs>
- **Vercel Support:** support@vercel.com
- **Railway Documentation:** <https://docs.railway.app>

- **Stripe Support:** <https://support.stripe.com>
-

Congratulations! 🎉 Your store is now live in production!

Next Steps:

- Marketing and SEO
- Add products
- Set up customer support
- Monitor analytics
- Gather customer feedback

Last Updated: November 2024